Directed Graph Hashing

Caleb Helbling*

Abstract. We present methods, with correctness proofs, for hashing node-labeled directed graphs and the individual nodes within them. We distinguish between hashing based on the *global* structure of a graph, where equal hashes correspond to isomorphism and automorphism orbits, and hashing based on *local* observations obtained by traversing the graph along its edges, where equal hashes correspond to isomorphic (possibly infinite) unfoldings or, in a variant, to bisimilarity. For graphs whose outgoing edges are ordered we give linear-time node hashes and a quadratic-time canonization of strongly connected graphs, and we combine these ingredients into Merkle-style hashes that recurse over the condensation of a graph, in a global variant and a polynomial-time local variant, characterizing exactly which nodes receive equal hashes under each. We also show that hashing nodes by the language of their walks is PSPACE-hard. Structural hashing of trees is ubiquitous in blockchains, version control, and content-addressed storage; by contrast, hashing of cyclic directed graphs has received comparatively little systematic treatment. A reference implementation is validated against brute-force equivalence checks, and we report its running time on random digraphs.

1 Introduction Hashing functions are fundamental building blocks in the modern software stack that have seen widespread use in data storage and retrieval, digital fingerprinting, checksum and cryptographic applications. Traditionally, hash functions take as input bit strings of arbitrary length and return a bit string of a fixed size $\mathcal{H} : \{0,1\}^* \to \{0,1\}^n$. Good hashing functions should be fast to compute, minimize the chance of duplication of output values, and produce large changes in output for small changes in input. Many bit string hashing functions have been created over the years, including well known algorithms such as MD5 [32], SHA-2 [28], and SHA-3 [29].

Algorithm 1.1 Recursive Merkle-tree Hashing of Ordered Binary Tree

$$\mathrm{mh}(n) = \begin{cases} \mathcal{H}(n.\mathrm{label}) & n \text{ is a leaf} \\ \mathcal{H}(n.\mathrm{label} \circ \mathrm{mh}(n.\mathrm{left}) \circ \mathrm{mh}(n.\mathrm{right})) & \text{otherwise} \end{cases}$$

Hashing functions can be extended to trees via a simple extension of Merkle hashing [25]. In Merkle tree hashing, the hash of a node is computed by hashing the concatenation of the node's label with the recursive hash of the node's children. Algorithm 1.1 gives a simple recursive implementation of Merkle tree hashing for ordered binary trees. This particular variation of Merkle hashing has seen use in many applications, including cryptocurrency [26], version control systems [20], and distributed hypertext document storage systems [1]. For *unordered* trees, replacing the sequence of child hashes by a sorted multiset of child hashes yields a hash that is invariant under tree isomorphism, in the spirit of the classical Aho–Hopcroft–Ullman tree isomorphism algorithm [3].

Extending hashing to node-labeled directed graphs is non-trivial due to the presence of cycles: the recursion of Algorithm 1.1 no longer terminates. Indeed, the design space for hashing digraphs is surprisingly broad and depends on use case objectives. In this paper we focus on the following approaches, which we think are the most useful for practical applications:

- **Global Hashing** (Section 3) — for use cases where the *global* structure of the digraph is of importance. Two graphs hash equally if and only if they are isomorphic, and two nodes hash equally if and only if some

*Draper (chelbling@draper.com).

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

isomorphism of their graphs maps one onto the other; in particular, nodes in the same automorphism orbit hash equally.

- **Local Hashing** (Section 4) — for use cases where observers can only observe *local* information by traversing from a node to a neighbor on an outgoing edge. Two nodes hash equally if and only if the (potentially infinite) trees obtained by unfolding the graph from them are isomorphic. We also give a variant for bisimilarity, and show that hashing by the language of walks is PSPACE-hard. Local hashing has many commonalities with bisimulation and co-inductive representations of finite directed graphs [33, 31].

- **Ordered Outgoing Edges** (Section 5) — in many practical applications the outgoing edges from a node are ordered. We use this additional structure to obtain linear-time node hashes and a quadratic-time canonization of strongly connected graphs.

- **Merkle Graph Hashing** (Section 6) — for use cases where graphs are hashed recursively, or are built up over time. We hash strongly connected components individually, either globally (up to rooted isomorphism) or locally (up to unfolding isomorphism), then recurse over the acyclic condensation graph. We characterize precisely which nodes receive equal hashes in each variant. Both equivalences lie strictly between rooted isomorphism and unfolding isomorphism, and the local variant runs in polynomial time even for unordered graphs.

One common theme throughout this paper is reliance on conversion of graphs to *canonical form*. If two graphs (or nodes within them) can be converted to a canonical form, we can serialize the canonical form and pass it through an ordinary string hashing function. We distinguish carefully between the hash of an entire graph and the hash of a specific node within a graph. Our general strategy for computing the hash of a specific node is to combine the overall graph hash with the node's position in canonical order, taking care that this position accounts for any symmetry present in the graph (two nodes may be identical modulo automorphism).

Contributions. Each hashing scheme comes with an exact characterization of hash equality (Theorems 3.1, 3.2, 4.11, 4.14, 5.2, 5.4, 6.2 and 6.4), stated relative to an idealized string hash (Section 2.4). We identify and prove the *local stabilization* property that makes color-refinement colors usable as a canonical order (Lemma 4.8), prove PSPACE-hardness of hashing walk languages (Proposition 4.12), characterize the local Merkle equivalence by SCC-annotated unfoldings (Proposition 6.5), and position both Merkle schemes relative to the other equivalences (Proposition 6.6). A reference implementation is validated against brute-force checks and benchmarked in Section 9.

2 Preliminaries

2.1 Labeled digraphs and symmetry. A *labeled digraph* is a triple $G = (V, E, L)$ where V is a finite vertex set, $E \subseteq V \times V$ is the edge set (self-loops are allowed), and L maps vertices to labels drawn from a set of strings. In Section 4 we also use *labeled multidigraphs* $G = (V, \mu, L)$, where $\mu : V \times V \to \mathbb{N}$ gives edge multiplicities; a digraph is the special case $\mu(u, v) = [(u, v) \in E]$. We write $\mathrm{succ}(u)$ for the multiset of successors of u (with v occurring $\mu(u, v)$ times), and $R(u)$ for the set of vertices reachable from u (including u). A set $S \subseteq V$ is *forward-closed* if every successor of a vertex of S lies in S; each $R(u)$ is forward-closed, and so is V.

An *isomorphism* $\phi : G_1 \to G_2$ is a bijection $V_1 \to V_2$ with $L_2(\phi(u)) = L_1(u)$ and $\mu_2(\phi(u), \phi(v)) = \mu_1(u, v)$ for all u, v; we write $G_1 \cong G_2$. A *rooted isomorphism* $(G_1, v_1) \cong (G_2, v_2)$ is an isomorphism with $\phi(v_1) = v_2$. An *automorphism* is an isomorphism $G \to G$; the automorphisms form the group

$$\mathrm{Aut}(G) = \{\sigma : V \to V \text{ bijective} \mid L(\sigma(u)) = L(u),\ (u, v) \in E \leftrightarrow (\sigma(u), \sigma(v)) \in E\}.$$

Informally, the automorphism group gives all of the ways that G is symmetrical. The *orbit* of a vertex is the set of vertices it can be mapped to by automorphisms,

$$\mathrm{Orb}(G, v) = \{\sigma(v) \mid \sigma \in \mathrm{Aut}(G)\},$$

and the orbits partition V. Figure 2.1 gives an example. Note that v_1 and v_2 lie in the same orbit of G if and only if $(G, v_1) \cong (G, v_2)$.

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

2.2 Canonical forms. Fix a canonical labeling procedure (for example nauty/Traces [23, 24], bliss [15], or saucy [12], all of which support vertex labels). Given G with $|V| = n$ it returns a bijection $\rho_G : V \to \{0, \dots, n-1\}$ such that the relabeled graph $\mathrm{canon}(G) = \rho_G(G)$ satisfies

$$\mathrm{canon}(G_1) = \mathrm{canon}(G_2) \iff G_1 \cong G_2.$$

By construction ρ_G is an isomorphism $G \to \mathrm{canon}(G)$. It is determined only up to automorphisms: if ρ is a canonical labeling then so is $\rho \circ \sigma$ for every $\sigma \in \mathrm{Aut}(G)$, and no canonical labeling procedure can prefer one of these over another. This is precisely why node hashes must be defined through orbits rather than through canonical indices alone.

2.3 Strongly connected components. A *strongly connected component* (SCC) of a digraph is a maximal set of vertices such that there is a path between any two of them. The *condensation graph* is formed by contracting each SCC into a single node; it is acyclic, and all SCCs together with the condensation can be computed in linear time [36]. The *height* of an SCC is the number of edges on a longest path starting from it in the condensation. Figure 2.2 shows an example.

2.4 The hashing model. All of our schemes serialize a finite structure and apply a string hash $\mathcal{H}$. We assume a serialization $\mathrm{enc}(\cdot)$ of nested tuples, lists, integers and strings that is *injective* (for example, prefixing every item with a type tag and its length, so that labels containing delimiter characters cannot cause ambiguity), and we encode a multiset as the sorted list of its elements. Since $\mathcal{H}$ maps into a finite set it cannot be injective, so we state our theorems in the following sense.

Definition 2.1 (Correctness up to collisions). A hash h *realizes* an equivalence relation $\sim$ if (i) $x \sim y$ implies $h(x) = h(y)$, and (ii) whenever $h(x) = h(y)$ but $x \not\sim y$, the computations of $h(x)$ and $h(y)$ contain two distinct strings $s \neq s'$ with $\mathcal{H}(s) = \mathcal{H}(s')$.

Condition (ii) says that a failure of the "only if" direction exhibits an explicit collision of $\mathcal{H}$, which is believed infeasible for SHA-2 or SHA-3. Equivalently, every statement "$h(x) = h(y)$ if and only if $x \sim y$" below is proved under the idealization that $\mathcal{H}$ is injective, and each proof of the "only if" direction uses injectivity only by comparing two strings whose hashes are equal. For readability we say "assuming $\mathcal{H}$ is injective" in the proofs.

3 Global Graph Hashing In global graph hashing, we would like our computed hash values to be unique modulo graph isomorphism: $\mathcal{H}_G(G_1) = \mathcal{H}_G(G_2)$ if and only if $G_1 \cong G_2$. We canonize the input digraph and construct an adjacency list sorted in canonical order. The node labels (in canonical order) and

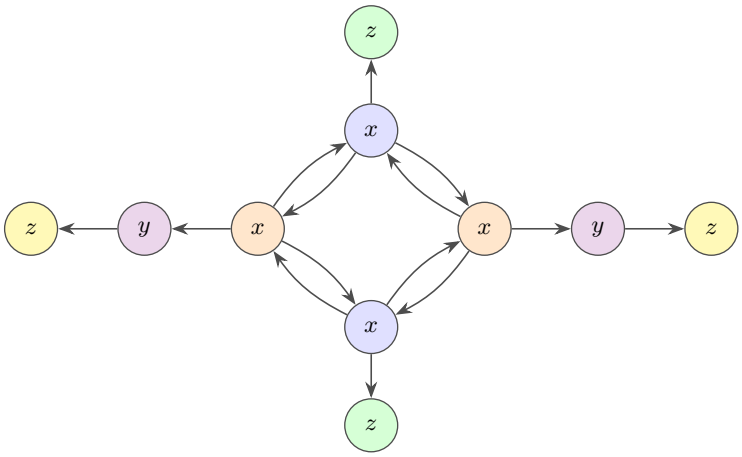


Figure 2.1: A directed graph with three node labels (x, y, z, shown as letters) and five automorphism orbits (shown by fill colour). Its automorphism group has order four and is generated by the two reflections of the drawing (their product is the rotation by 180°). Equal labels do not imply equal orbits: the four x-labelled nodes of the central diamond fall into two orbits, and the four z-labelled sinks fall into two orbits, because an x-node's pendant neighbour is a sink in one case and has a successor in the other.

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

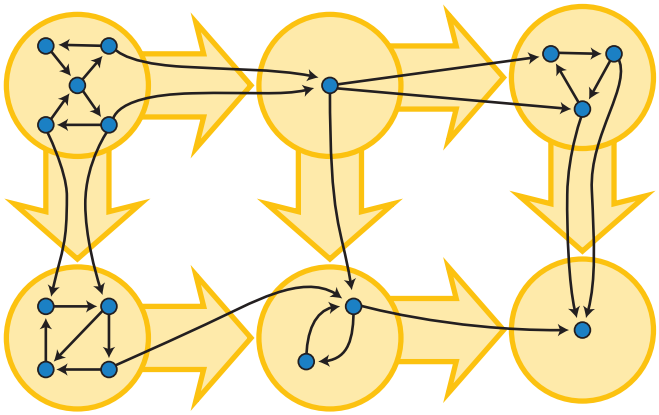


Figure 2.2: Each yellow node contains a strongly connected component of a graph. When each strongly connected component is contracted into a single node, the yellow condensation graph is formed. Since the condensation graph is acyclic, it can be recursed over. Public domain figure courtesy of David Eppstein.[2]

the canonical adjacency list are then serialized and hashed using a string hashing function $\mathcal{H}$. Algorithm 3.1 gives an implementation of this approach. Note that the pair (labels, adj_list) is exactly an encoding of canon(G): the number of vertices is the length of the label list, so isolated vertices are accounted for.

THEOREM 3.1 (Correctness of the global graph hash). *$\mathcal{H}_G$ realizes graph isomorphism: assuming $\mathcal{H}$ is injective, $\mathcal{H}_G(G_1) = \mathcal{H}_G(G_2)$ if and only if $G_1 \cong G_2$.*

Proof. ($\Rightarrow$) If $\mathcal{H}_G(G_1) = \mathcal{H}_G(G_2)$, then by injectivity of $\mathcal{H}$ and enc the canonical label lists and adjacency lists coincide, i.e. $\text{canon}(G_1) = \text{canon}(G_2)$. Hence $G_1 \cong G_2$.
($\Leftarrow$) If $G_1 \cong G_2$ then $\text{canon}(G_1) = \text{canon}(G_2)$, so the label lists and sorted adjacency lists coincide, and so do their encodings and hashes. ∎

Complexity. Since $\mathcal{H}_G$ decides isomorphism of labeled digraphs (in the idealized model), computing it is at least as hard as graph isomorphism (GI). No polynomial-time algorithm for GI is known; the best known worst-case bound is quasipolynomial, both for isomorphism testing [6] and for canonization [7]. Practical canonization tools based on individualization and refinement [23, 24, 15, 12] are extremely fast on most inputs, but have exponential worst-case running time on specially constructed families. The same remarks apply to computing orbits, which Algorithm 3.2 below requires; tools such as nauty return the orbit partition as a by-product of canonization.

The algorithm for hashing nodes builds on the algorithm for hashing graphs. First, the graph is canonized and the hash of the canonical graph is computed using Algorithm 3.1. Then we compute the *orbit index* of the input vertex: the smallest canonical index in the orbit of $\rho(v)$ in the canonical graph canon(G). (Equivalently, it is $\min\{\rho(u) \mid u \in \text{Orb}(G, v)\}$, since ρ maps orbits of G onto orbits of canon(G).) The orbit index and the graph hash are then combined and passed through a string hashing function. Algorithm 3.2 gives an implementation

[2] https://commons.wikimedia.org/wiki/File:Graph_Condensation.svg

Algorithm 3.1 Global Digraph Hash

```
1: function H_G(G = (V, E, L))
2:     ρ ← CANONIZE(G)                          ▷ canonical labeling ρ : V → {0, ..., |V| − 1}
3:     adj_list ← SORT([(ρ(u), ρ(v)) | (u, v) ∈ E])
4:     labels ← [L(ρ^{-1}(i)) for i = 0, ..., |V| − 1]
5:     return H(enc((labels, adj_list)))
6: end function
```

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

Algorithm 3.2 Global Digraph Node Hash

```
1: function H_N(G, v)
2:     ρ ← CANONIZE(G)                          ▷ the same ρ is used in the next line
3:     g_hash ← H_G(G)                          ▷ computed from ρ as in Algorithm 3.1
4:     orbit_idx ← min Orb(canon(G), ρ(v))
5:     return H(enc((orbit_idx, g_hash)))
6: end function
```

of this approach. The orbit index does not depend on which canonical labeling $\rho \circ \sigma$, $\sigma \in \mathrm{Aut}(G)$, the canonizer happens to return, since $\rho(\sigma(v))$ lies in the same orbit of $\mathrm{canon}(G)$ as $\rho(v)$.

THEOREM 3.2 (Correctness of the global node hash). *$\mathcal{H}_N$ realizes rooted isomorphism: assuming $\mathcal{H}$ is injective, $\mathcal{H}_N(G_1, v_1) = \mathcal{H}_N(G_2, v_2)$ if and only if there is an isomorphism $\phi : G_1 \to G_2$ with $\phi(v_1) = v_2$. In particular, two nodes of the same graph hash equally if and only if they lie in the same orbit.*

Proof. Let ρ_1, ρ_2 be the canonical labelings of G_1, G_2.

($\Leftarrow$) Since $G_1 \cong G_2$, $C := \mathrm{canon}(G_1) = \mathrm{canon}(G_2)$ and $\mathcal{H}_G(G_1) = \mathcal{H}_G(G_2)$ by Theorem 3.1. The map $\alpha = \rho_2 \circ \phi \circ \rho_1^{-1}$ is a composition of isomorphisms $C \to G_1 \to G_2 \to C$, hence $\alpha \in \mathrm{Aut}(C)$, and $\alpha(\rho_1(v_1)) = \rho_2(v_2)$. Thus $\rho_1(v_1)$ and $\rho_2(v_2)$ lie in the same orbit of C, so the orbit indices coincide, and so do the node hashes.

($\Rightarrow$) Equal node hashes imply, by injectivity, equal graph hashes and equal orbit indices. By Theorem 3.1 (and its proof) $\mathrm{canon}(G_1) = \mathrm{canon}(G_2) =: C$. Two orbits of C with the same minimum element are the same orbit, so there is $\alpha \in \mathrm{Aut}(C)$ with $\alpha(\rho_1(v_1)) = \rho_2(v_2)$. Then $\phi = \rho_2^{-1} \circ \alpha \circ \rho_1$ is an isomorphism $G_1 \to G_2$ with $\phi(v_1) = v_2$. ∎

4 Local Graph Hashing In comparison to the global hashing strategy which depends on the global structure of the graph, the local hashing strategy depends only on properties we can *observe* by traversing the digraph forward along its edges. To make this concept precise, the hashes of two nodes should be equal if and

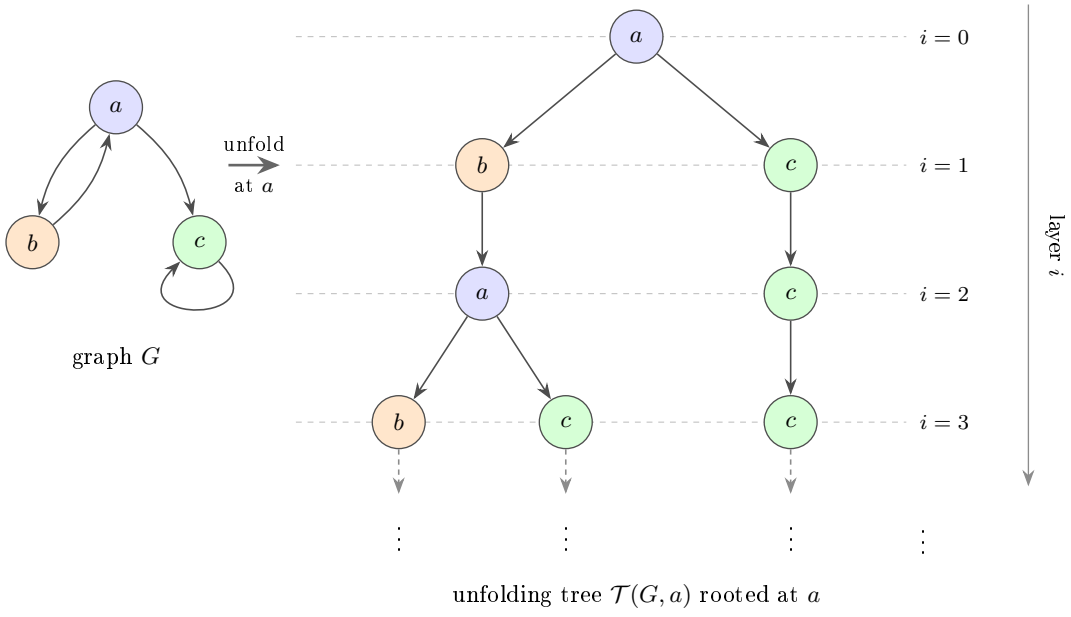


Figure 4.1: Unfolding of a directed graph with cycles. Starting from the root a, every vertex is replaced by one copy per walk reaching it, so the cycle $a \to b \to a$ and the self-loop $c \to c$ are unrolled into an infinite tree; only the layers $i = 0, \ldots, 3$ are drawn. Each tree node at layer i corresponds to a walk of length i from a in G; colours indicate the end vertex of the walk. The color $\lambda_i(a)$ is the (multiset) Merkle hash of the truncation $\mathcal{T}_i(G, a)$ consisting of layers $0, \ldots, i$.

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

only if their *unfoldings* are isomorphic.

4.1 Unfoldings.

Definition 4.1 (Unfolding). Let $G = (V, \mu, L)$ be a labeled multidigraph and $u \in V$. The *unfolding* $\mathcal{T}(G, u)$ is the rooted, labeled, unordered tree defined as follows. The nodes of $\mathcal{T}(G, u)$ are the walks in G starting at u, i.e. sequences $w = (x_0, e_1, x_1, \ldots, e_k, x_k)$ of vertices $x_j \in V$ of G with $x_0 = u$ and $e_j \in \{1, \ldots, \mu(x_{j-1}, x_j)\}$ selecting one of the parallel edges from x_{j-1} to x_j. Each walk, taken as a whole, is one tree node; the same vertex of G can therefore occur in many tree nodes. The root is the walk (u) of length 0, the parent of a walk of length $k \geq 1$ is obtained by deleting its last step (e_k, x_k), and the label of w is $L(x_k)$, the label of the vertex at which the walk ends. The *truncation* $\mathcal{T}_k(G, u)$ is the subtree of walks of length at most k.

Isomorphism of rooted labeled unordered trees means a label-preserving bijection preserving the root and the parent relation. Two finite such trees are isomorphic if and only if the roots have equal labels and there is a bijection between the children of the two roots that maps each child subtree to an isomorphic child subtree. We use two elementary facts. First, the subtree of $\mathcal{T}(G, u)$ below a walk ending at x is isomorphic to $\mathcal{T}(G, x)$. Second, the root of $\mathcal{T}(G, u)$ has exactly $\mu(u, x)$ children whose subtrees are copies of $\mathcal{T}(G, x)$, for each x. Unfoldings are infinite as soon as a cycle is reachable, but they are finitely branching. Figure 4.1 shows an example.

LEMMA 4.2 (Truncations suffice). *$\mathcal{T}(G_1, u) \cong \mathcal{T}(G_2, v)$ if and only if $\mathcal{T}_k(G_1, u) \cong \mathcal{T}_k(G_2, v)$ for every $k \geq 0$.*

Proof. ($\Rightarrow$) An isomorphism preserves depth, so it restricts to isomorphisms of the truncations. ($\Leftarrow$) Let I_k be the set of isomorphisms $\mathcal{T}_k(G_1, u) \to \mathcal{T}_k(G_2, v)$. Each I_k is finite (the trees are finite) and nonempty by assumption, and restriction maps I_{k+1} into I_k. Let $J_k \subseteq I_k$ be the set of elements of I_k that are restrictions of elements of I_j for every $j > k$. Since the images of I_j in I_k form a decreasing chain of finite nonempty sets, J_k is nonempty, and every element of J_k is the restriction of an element of J_{k+1}. By Kőnig's lemma [19] (or simply by induction) there is a sequence $\phi_k \in J_k$ with ϕ_{k+1} extending ϕ_k. Their union is an isomorphism $\mathcal{T}(G_1, u) \to \mathcal{T}(G_2, v)$. ∎

4.2 Color refinement. Our strategy for local graph hashing uses a variant of color refinement [38, 35] along outgoing edges. The initial coloring is the hash of the node label, and each round combines a node's label with the multiset of its successors' colors (a multiset is serialized by sorting):

$$\lambda_0(u) = \mathcal{H}(\mathrm{enc}(L(u))), \tag{4.1}$$

$$\lambda_{i+1}(u) = \mathcal{H}\big(\mathrm{enc}\big((L(u), \{\!\{\lambda_i(v) \mid v \in \mathrm{succ}(u)\}\!\})\big)\big). \tag{4.2}$$

Note that the colors are defined for nodes of *any* graph by the same formula, so colors of nodes in different graphs can be compared.

LEMMA 4.3. *Assuming $\mathcal{H}$ is injective, for nodes u of G_1 and v of G_2 and every $i \geq 0$: $\lambda_i(u) = \lambda_i(v)$ if and only if $\mathcal{T}_i(G_1, u) \cong \mathcal{T}_i(G_2, v)$. Consequently the colorings refine monotonically: $\lambda_{i+1}(u) = \lambda_{i+1}(v)$ implies $\lambda_i(u) = \lambda_i(v)$.*

Proof. Induction on i. For $i = 0$ both sides say $L_1(u) = L_2(v)$. For the step, by injectivity $\lambda_{i+1}(u) = \lambda_{i+1}(v)$ if and only if $L_1(u) = L_2(v)$ and the multisets $\{\!\{\lambda_i(x) \mid x \in \mathrm{succ}(u)\}\!\}$ and $\{\!\{\lambda_i(y) \mid y \in \mathrm{succ}(v)\}\!\}$ are equal. By the induction hypothesis the latter holds if and only if there is a bijection between the children of the two roots of $\mathcal{T}_{i+1}$ matching isomorphic depth-i subtrees, which (with the labels) is exactly $\mathcal{T}_{i+1}(G_1, u) \cong \mathcal{T}_{i+1}(G_2, v)$. Monotonicity follows because an isomorphism of $\mathcal{T}_{i+1}$ restricts to one of $\mathcal{T}_i$. ∎

Thus $\lambda_i(u)$ is exactly the multiset Merkle hash of the finite tree $\mathcal{T}_i(G, u)$, built bottom-up one layer per round; the bottom-up construction maximally shares computation in a dynamic programming fashion. Since colors change every round, we stop based on the *partition* they induce. For $S \subseteq V$ let $\pi_i(S)$ be the partition of S into classes of equal λ_i.

Stabilization on S: a set S is *stable in round* i if $\pi_{i+1}(S) = \pi_i(S)$. *Global stabilization* is stabilization on $S = V$. A node n is *locally stable* in round i if $R(n)$ is stable in round i.

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

By monotonicity, $\pi_{i+1}(S) = \pi_i(S)$ holds if and only if λ_i and λ_{i+1} take the same *number* of distinct values on S, which is how the check is implemented.

LEMMA 4.4 (Stability persists). *Assume $\mathcal{H}$ is injective and let S be forward-closed. If S is stable in round i, then $\pi_j(S) = \pi_i(S)$ for all $j \geq i$. Moreover, S is stable in some round $i \leq |S| - 1$.*

Proof. It suffices to show that stability in round j implies stability in round $j+1$. If $\pi_{j+1}(S) = \pi_j(S)$, the assignment $\beta(\lambda_j(x)) = \lambda_{j+1}(x)$ for $x \in S$ is a well-defined bijection between the color sets of rounds j and $j+1$ on S. Let $u, v \in S$. All successors of u and v lie in S because S is forward-closed, and the round-$(j+1)$ multiset of successor colors of u is the image under β of the round-j multiset. Since β is injective, the round-$(j+1)$ multisets of u and v are equal if and only if the round-j multisets are. By (4.2) and injectivity of $\mathcal{H}$, $\lambda_{j+2}(u) = \lambda_{j+2}(v)$ if and only if $\lambda_{j+1}(u) = \lambda_{j+1}(v)$, i.e. $\pi_{j+2}(S) = \pi_{j+1}(S)$. This completes the inductive step.

For the bound: by monotonicity each round either leaves $\pi_i(S)$ unchanged or strictly increases the number of classes, which is at most $|S|$; $\pi_0(S)$ has at least one class. ∎

COROLLARY 4.5. *Assume $\mathcal{H}$ is injective. Let S be forward-closed and stable in round i. For $u, v \in S$, $\lambda_i(u) = \lambda_i(v)$ if and only if $\mathcal{T}(G, u) \cong \mathcal{T}(G, v)$.*

Proof. If $\lambda_i(u) = \lambda_i(v)$ then $\lambda_j(u) = \lambda_j(v)$ for all $j \geq i$ by Lemma 4.4 and for all $j \leq i$ by monotonicity. By Lemma 4.3 all truncations are isomorphic, and by Lemma 4.2 so are the unfoldings. The converse follows from Lemma 4.3. ∎

4.3 The unfolding quotient. Write $u \equiv v$ if $\mathcal{T}(G, u) \cong \mathcal{T}(G, v)$. By Corollary 4.5 with $S = V$, the global stable coloring $C = \lambda_{i^*}$, where i^* is the first round of global stabilization, has exactly the classes of $\equiv$.

Definition 4.6 (Unfolding quotient). We define the quotient multidigraph $Q = G/\equiv$, written (V_Q, μ_Q, L_Q), as follows: its vertices are the classes of $\equiv$, its labels are $L_Q([u]) = L(u)$, and its multiplicities are

$$\mu_Q(A, B) = \sum_{x \in B} \mu(u, x) \quad \text{for any } u \in A,$$

i.e. the number of edges from a single representative of A into B.

This is *not* the naive quotient that creates one edge $[u] \to [v]$ for every edge (u, v) of G: that construction multiplies multiplicities by class sizes and does not preserve unfoldings. Definition 4.6 coincides with the minimum base of a graph fibration [10].

LEMMA 4.7. *Definition 4.6 is independent of the chosen representative; $\mathcal{T}(Q, [u]) \cong \mathcal{T}(G, u)$ for every $u \in V$; and distinct vertices of Q have non-isomorphic unfoldings.*

Proof. If $u \equiv u'$, an isomorphism $\mathcal{T}(G, u) \to \mathcal{T}(G, u')$ matches the root children, whose subtrees are the unfoldings of the successors. Hence for every $\equiv$-class B the number of successors (with multiplicity) of u in B equals that of u', so μ_Q is well defined. Next we show $\mathcal{T}_k(Q, [u]) \cong \mathcal{T}_k(G, u)$ by induction on k, simultaneously for all u. The case $k = 0$ holds by the choice of L_Q. For the step, the root of $\mathcal{T}_{k+1}(G, u)$ has, for each class B, exactly $\mu_Q([u], B)$ children whose subtrees are $\mathcal{T}_k(G, x)$ with $x \in B$. By the induction hypothesis each of these is isomorphic to $\mathcal{T}_k(Q, B)$, and the root of $\mathcal{T}_{k+1}(Q, [u])$ has exactly $\mu_Q([u], B)$ children with subtree $\mathcal{T}_k(Q, B)$. Lemma 4.2 gives the full unfoldings. Finally, if $A \neq B$ in V_Q with representatives $u \not\equiv v$, then $\mathcal{T}(Q, A) \cong \mathcal{T}(G, u) \not\cong \mathcal{T}(G, v) \cong \mathcal{T}(Q, B)$. ∎

4.4 Local stabilization and canonical order. We now run a fresh color refinement on Q (colors λ_i below refer to Q). For a vertex n of Q, let i_n be the first round in which n is locally stable, i.e. in which $R(n)$ is stable. We order $R(n)$ by the colors λ_{i_n} and serialize.

LEMMA 4.8 (Local colors are a canonical order). *Assume $\mathcal{H}$ is injective. For every vertex n of Q, λ_{i_n} is injective on $R(n)$.*

Proof. $R(n)$ is forward-closed and stable in round i_n. If $\lambda_{i_n}(x) = \lambda_{i_n}(y)$ for $x, y \in R(n)$, then $\mathcal{T}(Q, x) \cong \mathcal{T}(Q, y)$ by Corollary 4.5, so $x = y$ by Lemma 4.7. ∎

LEMMA 4.9 (Local colors are intrinsic). *Let $n_1 \in V_{Q_1}$, $n_2 \in V_{Q_2}$, and let ψ be a rooted isomorphism between the sub-multigraphs induced on $R(n_1)$ and $R(n_2)$, with $\psi(n_1) = n_2$. Then $\lambda_i(\psi(x)) = \lambda_i(x)$ for all i and all $x \in R(n_1)$, $i_{n_1} = i_{n_2}$, and* SERIALIZE *in Algorithm 4.1 produces the same string for n_1 and n_2.*

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

Proof. The first claim follows by induction on i from (4.1)–(4.2), since the successors of x lie in $R(n_1)$ and ψ preserves labels and multiplicities. Hence ψ maps $\pi_i(R(n_1))$ onto $\pi_i(R(n_2))$ for every i, so both sets first stabilize in the same round. Sorting by $\lambda_{i_{n_1}}$ therefore gives $\mathrm{index}(\psi(x)) = \mathrm{index}(x)$, so the root index, label lists and multiplicity lists coincide. ∎

To summarize the approach: we first merge all nodes with isomorphic unfoldings using color refinement with the global stabilization condition, obtaining the quotient multigraph Q in which distinct nodes have distinct unfoldings. We then refine again on Q, and for each node n we wait until its reachable set stabilizes. At that point the colors on $R(n)$ are pairwise distinct (Lemma 4.8) and depend only on $R(n)$ (Lemma 4.9), so they give a canonical order from which a canonical adjacency list is serialized and hashed. Algorithm 4.1 gives pseudo-code that computes the hash of every node in the input graph.

Remark 4.10 (Why local stabilization is needed). It is tempting to sort $R(n)$ by the global stable colors λ_{i^*} of Q instead. These are also injective on $R(n)$, but the round i^* depends on parts of the graph that are *not* reachable from n, and the color *values* (unlike the partition) keep changing from round to round. The same reachable subgraph embedded in two different graphs would then be sorted by colors from different rounds, generally in different orders, and the resulting hash would not be an invariant of the unfolding. Waiting for local stabilization makes the round i_n, and hence the order, intrinsic to $R(n)$.

THEOREM 4.11 (Correctness of the local hash). *$\mathcal{H}_L$ realizes unfolding isomorphism: assuming $\mathcal{H}$ is injective, for nodes u of G_1 and v of G_2, $\mathcal{H}_L(G_1)[u] = \mathcal{H}_L(G_2)[v]$ if and only if $\mathcal{T}(G_1, u) \cong \mathcal{T}(G_2, v)$.*

Proof. Let Q_1, Q_2 be the quotients, $A = [u] \in V_{Q_1}$ and $B = [v] \in V_{Q_2}$.

($\Rightarrow$) Equal hashes give, by injectivity, equal root indices, label lists and multiplicity lists. Mapping the k-th vertex of $R(A)$ in sorted order to the k-th vertex of $R(B)$ is therefore a rooted isomorphism of the induced sub-multigraphs. The unfolding of a vertex depends only on the sub-multigraph induced by its reachable set and is invariant under isomorphism, so $\mathcal{T}(Q_1, A) \cong \mathcal{T}(Q_2, B)$. By Lemma 4.7, $\mathcal{T}(G_1, u) \cong \mathcal{T}(G_2, v)$.

($\Leftarrow$) By Lemma 4.7 there is a tree isomorphism $\tau : \mathcal{T}(Q_1, A) \to \mathcal{T}(Q_2, B)$. Define $g : R(A) \to R(B)$ by

Algorithm 4.1 Local Digraph Node Hash

```
function SERIALIZE(Q = (V_Q, μ_Q, L_Q), root, reach, colors)
    order ← SORTED(reach, key=colors)                          ▷ no ties, by Lemma 4.8
    index ← {x ↦ i | (i, x) ∈ ENUMERATE(order)}
    adj_list ← SORTED([(index[s], index[t], μ_Q(s,t)) | s,t ∈ reach, μ_Q(s,t) > 0])
    labels ← [L_Q(x) for x in order]
    return H(enc((index[root], labels, adj_list)))
end function
function H_L(G = (V, E, L))
    C ← GLOBAL_STABLE_COLORS(G)                    ▷ refine with (4.2) until global stabilization
    Q = (V_Q, μ_Q, L_Q) ← G/C      ▷ unfolding quotient, Definition 4.6; C : V → V_Q is the projection
    reach ← {n ↦ R_Q(n) | n ∈ V_Q}
    final_hash ← EMPTY_DICT()
    λ ← {n ↦ H(enc(L_Q(n))) | n ∈ V_Q}
    while |final_hash| < |V_Q| do                  ▷ terminates by Lemma 4.4 with S = V_Q
        λ_next ← REFINE_ROUND(Q, λ)                          ▷ one application of (4.2) on Q
        for n ∈ V_Q with n ∉ final_hash do
            if |{λ(x) | x ∈ reach[n]}| = |{λ_next(x) | x ∈ reach[n]}| then      ▷ n locally stable
                final_hash[n] ← SERIALIZE(Q, n, reach[n], λ)
            end if
        end for
        λ ← λ_next
    end while
    return {u ↦ final_hash[C(u)] | u ∈ V}
end function
```

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

letting $g(X)$ be the end vertex of $\tau(w)$ for any walk w from A to X. The subtree below w is $\mathcal{T}(Q_1, X)$, and the subtree below $\tau(w)$ is $\mathcal{T}(Q_2, g(X))$, so these are isomorphic. By Lemma 4.7 (applied in Q_2), $g(X)$ does not depend on the choice of w. The same argument with τ^{-1} gives an inverse map, so g is a bijection with $g(A) = B$ and $\mathcal{T}(Q_1, X) \cong \mathcal{T}(Q_2, g(X))$ for all X. It preserves labels. For multiplicities, fix X and Z in $R(A)$. The root of $\mathcal{T}(Q_1, X)$ has $\mu_{Q_1}(X, Z)$ children of isomorphism type $\mathcal{T}(Q_1, Z)$. Since distinct vertices of Q_2 have distinct unfolding types, the root of $\mathcal{T}(Q_2, g(X))$ has $\mu_{Q_2}(g(X), g(Z))$ children of this type. An isomorphism between $\mathcal{T}(Q_1, X)$ and $\mathcal{T}(Q_2, g(X))$ preserves the number of children of each type, so $\mu_{Q_1}(X, Z) = \mu_{Q_2}(g(X), g(Z))$. Thus g is a rooted isomorphism, and Lemma 4.9 shows that the serializations, and hence the hashes, are equal. ∎

Complexity. Let $n = |V|$ and $m = |E|$. Each refinement round costs $O((n + m) \log n)$ for sorting successor multisets (plus hashing of strings of total length $O(n + m)$ digests), and by Lemma 4.4 at most n rounds are needed in each stage. The local stability checks cost $\sum_{x \in V_Q} |R(x)| = O(n^2)$ per round, and the serializations cost $O(n(n+m) \log n)$ in total. The overall running time is therefore $O(n^3 + nm \log n)$. The serializations alone have total size $\Theta(\sum_x (|R(x)| + |E(R(x))|))$, which can be $\Theta(n(n + m))$, so a quadratic term is inherent in any scheme that serializes each node's reachable subgraph explicitly. The global stage can be implemented in $O((n+m) \log n)$ time by partition refinement [11, 30, 8]. In our experiments (Section 9) the number of rounds is small and the running time grows quadratically.

4.5 Relation to prior work. Our approach draws on color refinement for undirected graphs, also known as the 1-dimensional Weisfeiler–Leman algorithm and the basis of Weisfeiler–Lehman subtree kernels [38, 35, 34], and on partition refinement for automata, such as Hopcroft's DFA minimization algorithm [14, 18]. The equivalence $\equiv$ and the quotient Q correspond to the fibration-theoretic notions of *views* and *minimum bases* [39, 10], and the bound of Lemma 4.4 parallels Norris's theorem that universal covers isomorphic to depth $n - 1$ are isomorphic [27]. Unfoldings of finite graphs are exactly the regular (rational) trees, for which unique maximally-shared representations were studied by Mauborgne [21]. To our knowledge, the local stabilization condition and its use to turn refinement colors into a canonical order that is intrinsic to each node's reachable subgraph are new.

4.6 Alternative formulations of local hashing. The strategy above hashes what an observer can see by traversing the graph forward along its directed edges. We note two other formulations.

Walk languages. Treat labels as letters of an alphabet and define the *vertex language* of u as the set of label sequences of walks starting at u:

$$\mathrm{Lang}_G(u) = \{L(u)\} \cup \bigcup_{v \in \mathrm{succ}(u)} \{L(u) \circ x \mid x \in \mathrm{Lang}_G(v)\}. \tag{4.3}$$

This resembles a nondeterministic finite automaton in which letters are emitted when visiting nodes rather than when traversing edges, and in which every state is accepting. Every vertex language is therefore prefix-closed.

PROPOSITION 4.12. *Deciding $\mathrm{Lang}_G(u) = \mathrm{Lang}_G(v)$ is PSPACE-complete. Consequently, unless* P = PSPACE*, there is no polynomial-time computable function h with $h(u) = h(v)$ if and only if $\mathrm{Lang}_G(u) = \mathrm{Lang}_G(v)$.*

Proof. Membership: a vertex language is the language of an NFA with one state per vertex plus an initial state, and NFA equivalence is in PSPACE. Hardness: we reduce from equivalence of NFAs in which all states are final, which is PSPACE-complete over alphabets of size at least two [17]. Let $M = (P, \Sigma, \delta, p_0, P)$ be such an NFA without ε-moves. Build a vertex-labeled graph with a root r labeled by a fresh letter $\#$ and one vertex t labeled a for each transition $t = (p, a, q) \in \delta$. Add edges $r \to t$ for every transition leaving p_0, and $t \to t'$ whenever $t = (p, a, q)$ and t' leaves q. Walks from r correspond exactly to runs of M from p_0, and every run is accepting, so $\mathrm{Lang}(r) = \{\#w \mid w \in L(M)\}$. The graph has $O(|\delta|^2)$ edges. Building the graphs for two such NFAs M_1, M_2 side by side gives roots with equal languages if and only if $L(M_1) = L(M_2)$. The final statement follows because h would decide the problem in polynomial time. ∎

Bisimulation. Nodes a and b (possibly in different graphs) are *bisimilar* if they are related by the largest relation $\mathcal{R}$ such that $a\mathcal{R}b$ implies: (a) $L(a) = L(b)$; (b) for every edge $a \to a'$ there is an edge $b \to b'$ with $a'\mathcal{R}b'$; and (c) for every edge $b \to b'$ there is an edge $a \to a'$ with $a'\mathcal{R}b'$ [33]. The existential quantifiers, in place of a bijection between successors, mean that bisimulation ignores duplicated bisimilar successors: bisimilar nodes may have different out-degrees. Unfolding isomorphism is the counting analogue of bisimilarity and implies it.

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

A tempting modification of Algorithm 4.1 is to keep the multiset refinement and merely collapse the quotient G/C to an ordinary digraph. *This is incorrect*, because multiset refinement separates bisimilar nodes that differ only in the number of equivalent successors; after collapsing, such nodes become distinct quotient vertices with identical colors, and sorting by color is no longer canonical.

Example 4.13. Let $u \to a_1$, $u \to b$, $b \to a_2$, $a_1 \to x$, $a_2 \to x$ and $a_2 \to x'$, where x and x' are sinks with the same label and a_1, a_2 have the same label. Then a_1 and a_2 are bisimilar. Multiset refinement separates them ($\{\!\{X\}\!\}$ versus $\{\!\{X, X\}\!\}$), and in the collapsed quotient they have equal colors forever. Sorting $R(u)$ then depends on how the tie between a_1 and a_2 is broken, and the two tie-breaks produce different adjacency lists (the edge from u points to one or the other), so the hash is not well defined.

The correct modification replaces the multisets in (4.2) by *sets* in *both* refinement stages: $\lambda^{\text{set}}_{i+1}(u) = \mathcal{H}(\text{enc}((L(u), \{\lambda^{\text{set}}_i(v) \mid v \in \text{succ}(u)\})))$. The global stage then computes bisimilarity, as in relational coarsest partition algorithms [16, 30]. The quotient is the ordinary digraph with an edge $A \to B$ if and only if some (equivalently, every) $u \in A$ has a successor in B, and SERIALIZE emits a simple adjacency list. Call the resulting hash $\mathcal{H}_B$.

THEOREM 4.14. *$\mathcal{H}_B$ realizes bisimilarity: assuming $\mathcal{H}$ is injective, $\mathcal{H}_B(G_1)[u] = \mathcal{H}_B(G_2)[v]$ if and only if u and v are bisimilar.*

Proof sketch. The proof follows that of Theorem 4.11, replacing $\mathcal{T}_i$-isomorphism by i-step bisimilarity $\approx_i$. With sets in place of multisets, Lemma 4.3 becomes: $\lambda^{\text{set}}_i(u) = \lambda^{\text{set}}_i(v)$ if and only if $u \approx_i v$. Lemma 4.2 becomes the fact that on finitely branching systems bisimilarity equals $\bigcap_i \approx_i$ [13]. Lemma 4.4 and Corollary 4.5 go through verbatim (an injective map on colors preserves set equality just as it preserves multiset equality). The bisimulation quotient is bisimilar to the original graph, and on it bisimilarity is the identity, which replaces Lemma 4.7. For the ($\Leftarrow$) direction, bisimilarity restricted to $R(A) \times R(B)$ in the two quotients is total and surjective: by induction along walks, every vertex reachable from A is bisimilar to one reachable from B and vice versa. It is functional and injective by minimality, and it maps edges onto edges by clauses (b) and (c). It is therefore a rooted isomorphism, and Lemma 4.9 applies unchanged. ∎

5 Outgoing Ordered Edge Graphs In many real world applications the outgoing edges of each node are ordered. Examples include the links of a web page, the fields of a record, and the arguments of a function call. We take advantage of this extra structure to create simpler and faster hashing algorithms. The algorithms in this section were inspired by the more general Scott graph canonization algorithm of Bloyet, Marteau and Frénod [9]. The basic idea is that, because the edges are ordered, a depth-first search from a given root visits the nodes in an order that is determined by the structure alone.

Model. An *ordered labeled digraph* assigns to each node u a label $L(u)$ and a finite *out-sequence* $s(u) = (s_1(u), \ldots, s_{d(u)}(u))$ of successor nodes; position k is the k-th *port* of u, and repeated entries (two links to the same page) are allowed. An *ordered isomorphism* $\phi : G_1 \to G_2$ is a label-preserving bijection with $s(\phi(u)) = (\phi(s_1(u)), \ldots, \phi(s_{d(u)}(u)))$ for all u. Ordered automorphisms, orbits and rooted ordered isomorphisms

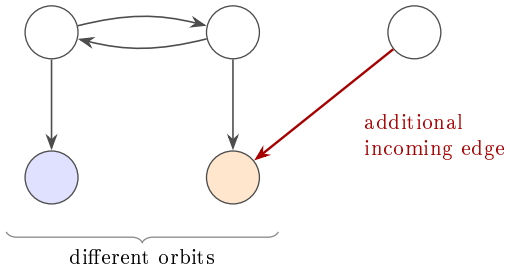


Figure 5.1: The symmetry of the two bottom nodes is broken by an incoming edge from elsewhere in the graph (all nodes carry the same label). Without the highlighted edge the two bottom nodes would be interchangeable; with it, no automorphism maps one onto the other, so they no longer lie in the same orbit. Their reachable subgraphs (a single sink each) are nevertheless identical, so any hash that looks only at reachable subgraphs must give them equal values.

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

are defined accordingly. For a node n we write $G[R(n)]$ for the ordered subgraph induced on its reachable set.

Example 5.1 (Edge order must be serialized). Sorting the relabeled adjacency list, as an earlier version of this algorithm did, loses the port order, and the depth-first numbering does not always recover it because edges to already-visited nodes do not affect the numbering. Take $s(r) = (x, y)$, $s(y) = ()$, and either $s(x) = (y, r)$ or $s(x) = (r, y)$. In both graphs the search from r numbers r, x, y as $0, 1, 2$, and the sorted edge set is $\{(0, 1), (0, 2), (1, 0), (1, 2)\}$ in both cases, even though the two graphs are not ordered-isomorphic.

Algorithm 5.1 Node Hash — Outgoing Ordered Edges

1. From the input node n, perform a depth-first search that explores the out-edges of each node in port order, and number the reachable nodes $0, 1, 2, \ldots$ in order of discovery (preorder).
2. Create the list of original labels in discovery order.
3. For each node in discovery order, write its out-sequence as a sequence of discovery numbers, *keeping the port order* (do not sort).
4. Return $\mathcal{H}_O(G, n) = \mathcal{H}(\text{enc}((\text{label list},\ \text{out-sequence list})))$.

THEOREM 5.2. *$\mathcal{H}_O$ realizes rooted ordered isomorphism of reachable subgraphs: assuming $\mathcal{H}$ is injective, $\mathcal{H}_O(G_1, n_1) = \mathcal{H}_O(G_2, n_2)$ if and only if there is an ordered isomorphism $G_1[R(n_1)] \to G_2[R(n_2)]$ mapping n_1 to n_2. It runs in time $O(|R(n)| + |E(R(n))|)$ plus the cost of hashing.*

Proof. ($\Leftarrow$) The depth-first search is deterministic given the port orders. An ordered isomorphism ψ commutes with every step of it: by induction on the steps, the searches from n_1 and $\psi(n_1) = n_2$ discover x and $\psi(x)$ at the same step. Hence ψ preserves discovery numbers, and the serializations coincide. ($\Rightarrow$) Equal serializations have equal lengths. Map the k-th discovered node of G_1 to the k-th discovered node of G_2. The label lists agree, and the out-sequences of corresponding nodes are equal as sequences of discovery numbers, so the map is an ordered isomorphism mapping n_1 (number 0) to n_2. ∎

$\mathcal{H}_O$ is a hash of the reachable subgraph, not of the unfolding. For example, a node with a self-loop and a node on a 2-cycle (with all labels equal) have isomorphic unfoldings but different values of $\mathcal{H}_O$. The local, unfolding-based notion is easier in the ordered setting than in Section 4, because an ordered graph is a deterministic automaton whose input letters are ports and whose output at each state is the pair (label, out-degree).

PROPOSITION 5.3 (Ordered local hashing). *Refine with sequences in place of multisets, $\lambda_{i+1}(u) = \mathcal{H}(\text{enc}((L(u), (\lambda_i(s_1(u)), \ldots, \lambda_i(s_{d(u)}(u))))))$, until global stabilization, and form the quotient Q with $s([u]) = ([s_1(u)], \ldots, [s_{d(u)}(u)])$. Then $u \mapsto \mathcal{H}_O(Q, [u])$ realizes isomorphism of ordered unfoldings. The quotient is the minimal automaton, and it can be computed in $O(m \log n)$ time with partial-DFA minimization [14, 37].*

Proof sketch. As in Lemmas 4.3–4.7, the stable classes are exactly the ordered-unfolding classes, the quotient is well defined and preserves ordered unfoldings, and distinct vertices of Q have distinct ordered unfoldings. If $\mathcal{T}(G_1, u) \cong \mathcal{T}(G_2, v)$ as ordered trees, then the simultaneous traversal of Q_1 from $[u]$ and Q_2 from $[v]$ along equal ports reaches vertices with isomorphic ordered unfoldings. By minimality this defines a bijection between the reachable sets, which is an ordered isomorphism, so Theorem 5.2 applies. The converse is as in Theorem 4.11. ∎

Strongly connected graphs. For the special case of a strongly connected ordered graph, the node hashes of Algorithm 5.1 yield both a canonical form and the orbit partition in polynomial time. This is the case needed by the global variant of the Merkle-style hashing of Section 6, where strongly connected components are hashed individually. (For the local variant, Section 6 serializes the quotient of Proposition 5.3 once per component instead of running a search from every node.)

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

Algorithm 5.2 Canonization and Orbit Partitioning for Strongly Connected Ordered Graphs

1. For each node n, compute $\mathcal{H}_O(G, n)$ using Algorithm 5.1. The orbits are the classes of equal $\mathcal{H}_O$ values.
2. Choose any node r with minimal $\mathcal{H}_O$ value. Its discovery order is the canonical labeling ρ, and its serialization is the canonical form $\mathrm{canon}(G)$.
3. The graph hash is $\mathcal{H}(\mathrm{enc}(\mathrm{canon}(G)))$, and the orbit index of v is $\min\{\rho(x) \mid \mathcal{H}_O(G, x) = \mathcal{H}_O(G, v)\}$. These are used exactly as in Algorithm 3.2.

THEOREM 5.4. *Let G_1, G_2 be strongly connected ordered digraphs with n nodes and m edges, and assume $\mathcal{H}$ is injective. Then: (a) $\mathcal{H}_O(G_1, u) = \mathcal{H}_O(G_2, v)$ if and only if there is an ordered isomorphism $\phi : G_1 \to G_2$ with $\phi(u) = v$; in particular, the classes of Step 1 are exactly the orbits; (b) $\mathrm{canon}(G_1) = \mathrm{canon}(G_2)$ if and only if $G_1 \cong G_2$, and ρ is an isomorphism onto $\mathrm{canon}(G)$; (c) the node hash of Step 3 realizes rooted ordered isomorphism, as in Theorem 3.2; (d) every ordered automorphism is determined by the image of a single node, so $|\mathrm{Aut}(G)| \le n$ and all orbits have equal size; (e) Algorithm 5.2 runs in $O(n(n+m))$ time plus the cost of hashing.*

Proof. (a) Since G_i is strongly connected, $R(u) = V_1$ and $R(v) = V_2$, so this is Theorem 5.2. (b) If $\phi : G_1 \to G_2$ is an isomorphism, then by (a) ϕ maps nodes to nodes with equal $\mathcal{H}_O$ values. The minimal values therefore coincide, and all minimizing nodes have the same serialization, which is determined by the hash. Conversely, equal serializations yield an isomorphism as in Theorem 5.2. The discovery order is exactly the bijection onto the serialized graph. (c) With (b) in place of the canonizer, the proof of Theorem 3.2 applies verbatim; the orbit index of Step 3 is the minimum canonical index over the orbit by (a). (d) An automorphism commutes with the depth-first search from any node u, which reaches all nodes, so it is determined by $\sigma(u)$. Hence $\mathrm{Aut}(G)$ acts semiregularly. (e) The algorithm runs n searches of cost $O(n + m)$ each. □

Why only strongly connected graphs? For a general digraph, the orbit of a node depends not only on the structure of its reachable subgraph but also on its ancestors. Incoming edges can break the symmetry of an orbit, as demonstrated in Figure 5.1, so the reachable-subgraph hashes of Algorithm 5.1 can be equal for nodes in different orbits. In a strongly connected graph this cannot happen, because every node reaches every other node. We do not address canonization of general (not strongly connected) ordered digraphs here, as the Merkle construction below only needs strongly connected components.

6 Merkle Graph Hashing For applications where graphs are built up over time, a recursive approach similar to Merkle-tree hashing is highly beneficial. We now construct such an algorithm, which recurses over the condensation graph. The condensation is acyclic, so the recursion is well founded.

To compute the hash of each node, we process the strongly connected components in reverse topological order, so that every edge leaving a component points to nodes that have already been hashed. Each node of the component is relabeled by combining its label with the hashes of its external targets: a multiset of hashes for unordered graphs; for ordered graphs, the out-sequence in which every internal target is replaced by a placeholder $\star$, so that the positions of internal and external links are both recorded. Finally the relabeled component, restricted to its internal edges, is hashed with a *component hash*. We consider two choices.

- In the *global variant* $\mathcal{H}_M^{\mathrm{glob}}$, the component hash is the global node hash (Algorithm 3.2) or, for ordered graphs, the node hash of Algorithm 5.2. It distinguishes nodes up to rooted isomorphism of their relabeled components.
- In the *local variant* $\mathcal{H}_M^{\mathrm{loc}}$, the component hash is the local component hash of Algorithm 6.2 below. It distinguishes nodes only up to isomorphism of their unfoldings inside the relabeled components, and it runs in polynomial time also for unordered graphs.

Algorithm 6.1 gives the pseudo-code; each component is processed once.

6.1 A local component hash. In a strongly connected graph every node reaches every other node. The local hash of Section 4 therefore simplifies: all vertices of the quotient become locally stable in the same round, and the reachable set of every vertex is the whole quotient, so the quotient is serialized once instead of once per node. Algorithm 6.2 handles unordered graphs and, with the sequence refinement of Proposition 5.3, ordered

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

Algorithm 6.1 Merkle Graph Node Hashing

function $\mathcal{H}_M(G = (V, E, L),$ COMP_HASH$)$
  $h \leftarrow$ EMPTY_DICT$()$
  for each SCC S in reverse topological order **do** ▷ Tarjan's algorithm; sinks first
    for $x \in S$ **do**
      if outgoing edges are unordered **then**
        $X(x) \leftarrow \{\!\{h[y] \mid y \notin S \wedge (x, y) \in E\}\!\}$
      else
        $X(x) \leftarrow [\,(h[y]$ **if** $y \notin S$ else $\star)$ **for** y **in** $s(x)\,]$
      end if
      $L'(x) \leftarrow \mathcal{H}(\mathrm{enc}((L(x), X(x))))$
    end for
    $G' \leftarrow (S,\ E \cap (S \times S),\ L')$ ▷ ordered case: keep internal targets in port order
    $h \leftarrow h \cup$ COMP_HASH(G') ▷ hashes of all $x \in S$; see the two variants above
  end for
  return h
end function

Algorithm 6.2 Local Component Hash

function LOCAL_COMP_HASH(S) ▷ S strongly connected
  $C \leftarrow$ GLOBAL_STABLE_COLORS(S) ▷ (4.2); ordered: sequences as in Proposition 5.3
  $Q = (V_Q, \mu_Q, L_Q) \leftarrow S/C$ ▷ Definition 4.6; ordered: quotient of Proposition 5.3
  $\lambda \leftarrow$ GLOBAL_STABLE_COLORS(Q) ▷ fresh refinement on Q, as in Algorithm 4.1
  order $\leftarrow$ SORTED$(V_Q,$ key$= \lambda)$ ▷ no ties, by Lemma 6.1
  index $\leftarrow \{X \mapsto i \mid (i, X) \in$ ENUMERATE(order)$\}$
  labels $\leftarrow [L_Q(X)$ **for** X **in** order$]$
  edges $\leftarrow$ SORTED$([(\mathrm{index}[X], \mathrm{index}[Y], \mu_Q(X, Y)) \mid \mu_Q(X, Y) > 0])$
    ▷ ordered: $[(\mathrm{index}[Y] \mid Y \in s(X))$ **for** X **in** order$]$
  $g \leftarrow \mathcal{H}(\mathrm{enc}((\mathrm{labels}, \mathrm{edges})))$
  return $\{x \mapsto \mathcal{H}(\mathrm{enc}((\mathrm{index}[C(x)], g))) \mid x \in V_S\}$
end function

graphs. Here GLOBAL_STABLE_COLORS refines as in Section 4 and returns the colors of the first round in which the partition of all vertices is stable.

LEMMA 6.1. *Assume $\mathcal{H}$ is injective, let S be strongly connected, and let Q be its quotient (unordered or ordered). Then Q is strongly connected, the colors λ computed in Algorithm 6.2 are pairwise distinct on V_Q, and Q has no automorphism other than the identity.*

Proof. Let (x, y) be an edge of S. Choosing x as the representative in Definition 4.6 shows that $\mu_Q(C(x), C(y)) \geq \mu(x, y) > 0$. Hence the projection maps walks of S to walks of Q, and since it is surjective, Q is strongly connected. Thus $R_Q(X) = V_Q$ for every vertex X, so the refinement on Q stops exactly in the round i_X of Section 4, and Lemma 4.8 shows that λ is injective on V_Q. An automorphism of Q preserves unfoldings, and distinct vertices of Q have non-isomorphic unfoldings by Lemma 4.7, so it is the identity. In the ordered case the same arguments apply to the quotient of Proposition 5.3. ∎

In particular, no orbit computation is needed in the local variant: the canonical index of a vertex of Q identifies it uniquely.

THEOREM 6.2 (Correctness of the local component hash). *Let S_1, S_2 be strongly connected (unordered or ordered) digraphs, let $x_1 \in V_{S_1}$ and $x_2 \in V_{S_2}$, and write $h_S =$ LOCAL_COMP_HASH(S). Assuming $\mathcal{H}$ is injective, $h_{S_1}(x_1) = h_{S_2}(x_2)$ if and only if $\mathcal{T}(S_1, x_1) \cong \mathcal{T}(S_2, x_2)$ (as ordered trees in the ordered case). Algorithm 6.2 runs in $O(k(n+m)\log n)$ time plus the cost of hashing, where n and m are the numbers of vertices and edges of S and $k \leq 2n$ is the total number of refinement rounds.*

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

Proof. Let Q_1, Q_2 be the quotients, $A = C_1(x_1)$ and $B = C_2(x_2)$.

($\Leftarrow$) By Lemma 4.7, $\mathcal{T}(Q_1, A) \cong \mathcal{T}(S_1, x_1) \cong \mathcal{T}(S_2, x_2) \cong \mathcal{T}(Q_2, B)$. As in the proof of Theorem 4.11 (respectively, Proposition 5.3) there is a rooted isomorphism ψ between the sub-multigraphs induced on $R(A)$ and $R(B)$ with $\psi(A) = B$. By Lemma 6.1 these are all of Q_1 and Q_2, so ψ is an isomorphism $Q_1 \to Q_2$. By Lemma 4.9, ψ preserves the colors of every round, so both refinements stop in the same round and $\mathrm{index}[\psi(X)] = \mathrm{index}[X]$ for every X. Hence the label lists, the edge lists, the component hashes g and the indices of A and B coincide.

($\Rightarrow$) By injectivity, $\mathrm{index}[A] = \mathrm{index}[B]$ and the label and edge lists coincide. Mapping the k-th vertex of Q_1 to the k-th vertex of Q_2 is then an isomorphism $Q_1 \to Q_2$ (it preserves labels and multiplicities, respectively out-sequences) that maps A to B. Hence $\mathcal{T}(Q_1, A) \cong \mathcal{T}(Q_2, B)$, and $\mathcal{T}(S_1, x_1) \cong \mathcal{T}(S_2, x_2)$ by Lemma 4.7.

For the running time, each refinement round costs $O((n+m)\log n)$, each of the two refinements needs at most n rounds by Lemma 4.4, and the serialization costs $O((n+m)\log n)$. ∎

The proof uses the colors of line 4 only through two properties: they are injective on V_Q, and isomorphisms between quotients preserve them. Both also hold for the coarsest stable coloring computed by canonical color refinement [8], which supports directed and edge-colored graphs (ports can be encoded as edge colors) and runs in $O((n+m)\log n)$ time. Together with partition refinement for line 2 [11, 30, 37], this gives an $O((n+m)\log n)$ bound for Algorithm 6.2, plus the cost of hashing.

6.2 Which nodes receive equal hashes? Within a component, the global variant respects the full structure, including automorphisms and incoming edges inside the component, while the local variant only sees what can be observed by following edges. Across components both behave like a Merkle tree: they compare external targets only through their hashes and ignore whether two targets are shared. We make this precise with a single definition that covers both variants. Fix labeled digraphs G_1 and G_2 (possibly equal), and let $W = V_1 \sqcup V_2$. For $u \in W$ let $S(u)$ be the SCC of u in its own graph, and let $\mathrm{ht}(u)$ be the height of $S(u)$.

Definition 6.3 (Merkle equivalences). Let $\sim$ be either rooted isomorphism or isomorphism of unfoldings (for ordered graphs: rooted ordered isomorphism or isomorphism of ordered unfoldings). We define a relation $\approx$ on W by well-founded recursion on $\max(\mathrm{ht}(u), \mathrm{ht}(v))$. Suppose that $\approx$ has been defined on the nodes of height less than h. For a node x of height at most h, all external successors of x have height less than h. Let $X^{\approx}(x)$ be the multiset of their $\approx$-classes; for ordered graphs, let it be the sequence whose k-th entry is the $\approx$-class of $s_k(x)$ if $s_k(x) \notin S(x)$, and $\star$ otherwise. Let $C^{\approx}(x)$ be the subgraph induced on $S(x)$ (for ordered graphs, keeping the internal targets of each node in port order), with labels $L^{\approx}(y) = (L(y), X^{\approx}(y))$. For u, v with $\max(\mathrm{ht}(u), \mathrm{ht}(v)) = h$, set

$$u \approx v \iff (C^{\approx}(u), u) \sim (C^{\approx}(v), v).$$

We write $\approx_{\mathrm{glob}}$ and $\approx_{\mathrm{loc}}$ for the relations obtained when $\sim$ is rooted isomorphism and isomorphism of unfoldings, respectively.

The recursion is well founded because external successors lie in SCCs of strictly smaller height. By induction on h, both relations are equivalence relations that relate only nodes of equal height. Indeed, if $u \approx v$ then every node of $S(u)$ corresponds to a node of $S(v)$ with the same label $L^{\approx}$ and vice versa (for unfoldings, because every node of a strongly connected graph occurs in its unfolding). So the two components have the same classes of external successors, and hence the same height. Spelled out, $u \approx_{\mathrm{glob}} v$ if and only if there is a bijection $\phi : S(u) \to S(v)$ with $\phi(u) = v$ that is a label-preserving isomorphism of the induced subgraphs and, for every $x \in S(u)$, matches the external successors of x bijectively with $\approx_{\mathrm{glob}}$-equivalent external successors of $\phi(x)$. For ordered graphs the matching is port by port, with internal ports mapped by ϕ.

THEOREM 6.4 (Correctness of the Merkle hashes). *Assuming $\mathcal{H}$ is injective, $\mathcal{H}_M^{\mathrm{glob}}$ realizes $\approx_{\mathrm{glob}}$ and $\mathcal{H}_M^{\mathrm{loc}}$ realizes $\approx_{\mathrm{loc}}$, for unordered and for ordered graphs. That is, for each variant, $\mathcal{H}_M(G_1)[u] = \mathcal{H}_M(G_2)[v]$ if and only if $u \approx v$ for the corresponding relation.*

Proof. By induction on $h = \max(\mathrm{ht}(u), \mathrm{ht}(v))$. By the induction hypothesis (vacuous when $h = 0$), two nodes of height less than h have equal hashes if and only if they are $\approx$-equivalent. So $[y]_{\approx} \mapsto h[y]$ is a well-defined injective map on these classes. Hence, for nodes x, x' of height at most h, $X(x) = X(x')$ if and only if $X^{\approx}(x) = X^{\approx}(x')$, and by injectivity $L'(x) = L'(x')$ if and only if $L^{\approx}(x) = L^{\approx}(x')$. Relabeling by an injective map preserves and reflects both rooted isomorphism and isomorphism of unfoldings. Therefore the relabeled components G' of Algorithm 6.1 containing u and v satisfy $(G'(u), u) \sim (G'(v), v)$ if and only if $(C^{\approx}(u), u) \sim (C^{\approx}(v), v)$. In the ordered case, L'

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

records the $\star$-masked sequence and G' keeps internal targets in port order, so together they determine every port, exactly as $C^{\approx}$ does. Finally, the component hash realizes $\sim$ on strongly connected graphs. For the global variant this is Theorem 3.2, or Theorem 5.4(c) for ordered graphs. For the local variant it is Theorem 6.2. ∎

The local equivalence has a direct description in terms of unfoldings. The *SCC-annotated unfolding* $\mathcal{T}^{\mathrm{scc}}(G,u)$ is the unfolding $\mathcal{T}(G,u)$ (ordered by port for ordered graphs) in which every tree edge, corresponding to a step from x_{k-1} to x_k, is marked *internal* if $x_k \in S(x_{k-1})$ and *external* otherwise. Isomorphisms of annotated unfoldings must preserve the marks.

PROPOSITION 6.5. *For nodes u of G_1 and v of G_2, $u \approx_{\mathrm{loc}} v$ if and only if $\mathcal{T}^{\mathrm{scc}}(G_1,u) \cong \mathcal{T}^{\mathrm{scc}}(G_2,v)$. The same holds for ordered graphs.*

Proof. Marks depend only on the endpoints of each step, so the subtree of $\mathcal{T}^{\mathrm{scc}}(G,u)$ below a walk ending at y is $\mathcal{T}^{\mathrm{scc}}(G,y)$. The walks from u that use only internal steps stay in $S(u)$. They form a subtree, the *internal part*, which is the unfolding at u of the subgraph induced on $S(u)$. A tree node of the internal part ending at x has one internal child for each internal edge of x and one external child for each external edge. We proceed by induction on $\max(\mathrm{ht}(u),\mathrm{ht}(v))$.

($\Rightarrow$) Let τ be an isomorphism of the unfoldings of $C^{\approx}(u)$ and $C^{\approx}(v)$, i.e. of the internal parts, preserving $L^{\approx}$. For a tree node w ending at x with $\tau(w)$ ending at x', we have $L(x) = L(x')$ and $X^{\approx}(x) = X^{\approx}(x')$. So there is a bijection (port by port for ordered graphs) between the external successors of x and those of x' relating only $\approx_{\mathrm{loc}}$-equivalent nodes, whose annotated unfoldings are isomorphic by the induction hypothesis. Extending τ at every external child by these isomorphisms gives an isomorphism $\mathcal{T}^{\mathrm{scc}}(G_1,u) \to \mathcal{T}^{\mathrm{scc}}(G_2,v)$.

($\Leftarrow$) An isomorphism σ of the annotated unfoldings maps internal children to internal children, so it restricts to an isomorphism τ of the internal parts preserving L. For a tree node w ending at x with $\sigma(w)$ ending at x', σ maps the external children of w bijectively (port by port) onto those of $\sigma(w)$, with isomorphic annotated subtrees. By the induction hypothesis the corresponding external successors are $\approx_{\mathrm{loc}}$-equivalent, so $X^{\approx}(x) = X^{\approx}(x')$. Thus τ preserves $L^{\approx}$, and $u \approx_{\mathrm{loc}} v$. ∎

PROPOSITION 6.6. *For nodes u of G_1 and v of G_2,*

$$(G_1[R(u)],u) \cong (G_2[R(v)],v) \;\Rightarrow\; u \approx_{\mathrm{glob}} v \;\Rightarrow\; u \approx_{\mathrm{loc}} v \;\Rightarrow\; \mathcal{T}(G_1,u) \cong \mathcal{T}(G_2,v),$$

and none of the converse implications holds. The same statements hold in the ordered setting.

Proof. A rooted isomorphism of reachable subgraphs maps SCCs to SCCs, so it witnesses Definition 6.3 for $\approx_{\mathrm{glob}}$ at every level. For the second implication, proceed by induction on height. If ϕ is a rooted isomorphism witnessing $u \approx_{\mathrm{glob}} v$, then ϕ preserves the labels $L^{\approx}$ defined with $\approx_{\mathrm{loc}}$ as well. This holds because, by the induction hypothesis, a matching of external successors by $\approx_{\mathrm{glob}}$-equivalent nodes is also a matching by $\approx_{\mathrm{loc}}$-equivalent nodes. A rooted isomorphism induces an isomorphism of unfoldings, so $u \approx_{\mathrm{loc}} v$. The third implication follows from Proposition 6.5 by forgetting the marks.

For strictness, all nodes in the following examples carry the same label.

- *First implication.* Let $x \to p$, $x \to q$, $p \to s$ and $q \to s$, and compare with the same graph in which s is replaced by two copies s_1, s_2 with $p \to s_1$ and $q \to s_2$. The nodes x are $\approx_{\mathrm{glob}}$-equivalent, since all SCCs are singletons and the external multisets agree, but the reachable subgraphs have 4 and 5 nodes.

- *Second implication.* A node with a self-loop and a node on a 2-cycle are $\approx_{\mathrm{loc}}$-equivalent, since both internal unfoldings are infinite paths without external edges. They are not $\approx_{\mathrm{glob}}$-equivalent, since their SCCs have different sizes.

- *Third implication.* A node u with a self-loop, and a node v whose only edge leads to a node w with a self-loop, have isomorphic unfoldings (infinite paths). They are not $\approx_{\mathrm{loc}}$-equivalent, because the first step from u is internal and the first step from v is external.

In the ordered setting the same examples apply. A further example inside a single strongly connected graph is $s(a) = (b,a)$, $s(b) = (a,a)$. Here $a \approx_{\mathrm{loc}} b$, since all nodes have the same label and out-degree, but a and b lie in different orbits, since an automorphism exchanging them would map $s(a) = (b,a)$ to $(a,b) \neq s(b)$. ∎

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

Remark 6.7 (Why the internal/external marks remain). In both variants, a node x that lies on no cycle is hashed as $F(L(x), \text{successor hashes})$ for a function F built from $\mathcal{H}$. No scheme of this kind can realize plain unfolding isomorphism. In the third example above, unfolding isomorphism would require $h(v) = h(w)$, while $h(v) = F(L, \{\!\{h(w)\}\!\})$. So computing $h(w)$ would produce a fixed point of $y \mapsto F(L, \{\!\{y\}\!\})$, which is infeasible if $\mathcal{H}$ behaves like a random function. The marks are thus the price of Merkle-style compositionality. Plain unfolding isomorphism is realized by the hashes of Section 4 and Proposition 5.3, but these are not computed compositionally over the condensation.

Thus both Merkle hashes deliberately forget whether subgraphs below a component are shared, exactly as Merkle trees and Merkle DAGs do in git and IPFS. The global variant retains the full structure inside each cycle; it suits applications that must distinguish, for example, a single self-referencing node from a cycle of identical nodes. The local variant retains only what can be observed by following edges, together with whether each edge leaves its component. It is much faster (Section 9), it merges nodes that no traversal can tell apart, and it is the variant used in Section 7.

Complexity and incrementality. Computing the SCCs takes $O(n+m)$ time, and each component is relabeled and hashed once. The total cost is $O((n+m)\log n)$ for the relabeling plus $\sum_S c(S)$, where $c(S)$ is the cost of the component hash on S, plus the cost of hashing. For the global variant, $c(S)$ is the cost of Algorithm 3.2 on S (GI-hard in general) or, for ordered graphs, $O(|S|(|S| + m_S))$ for Algorithm 5.2. For the local variant, $c(S) = O(k_S(|S| + m_S)\log|S|)$ by Theorem 6.2, where $k_S \le 2|S|$ is the number of refinement rounds, and $c(S) = O((|S|+m_S)\log|S|)$ with partition refinement. The local variant is therefore polynomial also for unordered graphs, and near-linear when few refinement rounds are needed. As with Merkle trees, changing a node changes only the hashes of nodes from which it is reachable. Adding a new node whose edges point only to existing nodes creates a new singleton SCC and leaves all existing hashes unchanged.

7 Application Sketch From the algorithms given so far, it may not be obvious how to implement these graph theoretic procedures for real world data. Here we show how to do so for a static hyperlinked web page system stored in a content addressed distributed hash table (DHT).

In this sketch, we assume that web pages are static HTML files that link to each other and to external pages, and that external pages are themselves published in the same content-addressed way. (An external page that links back into our site would belong to the same SCC and be hashed together with it.) The pages form an ordered digraph in which each page is a node, the links are edges, and the out-edges of a node are ordered by their position in the HTML file. Figure 7.1 gives a concrete example of the process described here.

We first compute the SCCs of the link graph and process them sinks first, as in Algorithm 6.1. Within a page, every link to another page of the same SCC is replaced by its internal port number, and every link leaving the SCC is replaced by the recursively computed hash of its target. The resulting text is the relabeled node label L' of the ordered variant. We use the local variant of Section 6. For each SCC, Algorithm 6.2 computes the ordered quotient, which merges pages that cannot be told apart by following links, together with its canonical order. We then insert the following items into the distributed hash table:

- The strongly connected component as a whole:
  - **Value**: the canonical form of the quotient, i.e. the relabeled page contents in canonical order together with each page's internal out-sequence in canonical indices.
  - **Key**: the string hash $\mathcal{H}$ of the value.
- For each page, a reference to a specific node of the quotient:
  - **Value**: the canonical index of the page's class in the quotient, together with the key of its SCC.
  - **Key**: the string hash $\mathcal{H}$ of the value. This is the page's hash.

By Theorem 6.4 and Proposition 6.5, two pages receive the same key exactly when their SCC-annotated ordered unfoldings are isomorphic. In other words, no sequence of clicks can tell them apart, except by observing which links leave their SCC. In particular, pages in the same orbit share a key, and so do duplicated pages. This is intended: the store is content-addressed, and such pages are indistinguishable by navigation. No orbit computation is needed, because quotients of strongly connected graphs have no non-trivial automorphisms (Lemma 6.1). The

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

Row 1: Original HTML Pages

index.html

```
<body>
 <h1>Welcome</h1>
 <a href="about.html">
  About Us
 </a>
 <a href="http://e.com">
  External Resource
 </a>
</body>
```

about.html

```
<body>
 <h1>About Us</h1>
 <a href="index.html">
  Home
 </a>
 <a href="contact.html">
  Contact
 </a>
 <a href="http://p.com">
  Partner Site
 </a>
</body>
```

contact.html

```
<body>
 <h1>Contact Us</h1>
 <a href="about.html">
  About
 </a>
 <a href="http://e.com">
  External Resource
 </a>
</body>
```

Row 2: Replace intra-SCC links with their internal port numbers, and inter-SCC links with recursively computed hashes

index.html

```
<body>
 <h1>Welcome</h1>
 <a href="{0}">
  About Us
 </a>
 <a href="#8ab43f">
  External Resource
 </a>
</body>
```

about.html

```
<body>
 <h1>About Us</h1>
 <a href="{0}">
  Home
 </a>
 <a href="{1}">
  Contact
 </a>
 <a href="#f2d91c">
  Partner Site
 </a>
</body>
```

contact.html

```
<body>
 <h1>Contact Us</h1>
 <a href="{0}">
  About
 </a>
 <a href="#8ab43f">
  External Resource
 </a>
</body>
```

Row 3: Construct strongly connected ordered digraph (labels taken from previous row)

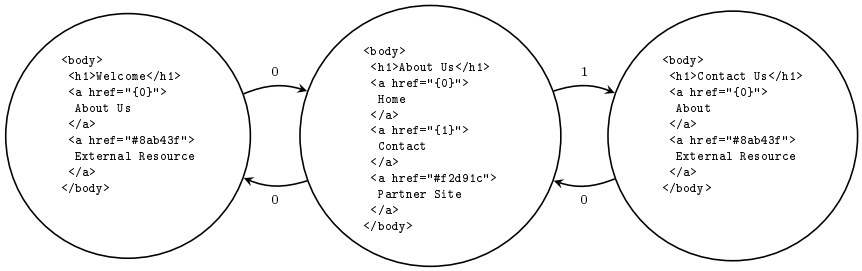


Figure 7.1: Hashing a three page website that forms a single SCC. In Row 2, {k} marks a page's k-th intra-SCC link (the placeholder $\star$ of Algorithm 6.1), and #8ab43f, #f2d91c are recursively computed hashes of external pages.

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

whole site is hashed in near-linear time in practice (Section 9). If instead the exact page structure of every SCC must be preserved, the global variant can be used: Algorithm 5.2 replaces the quotient by the full SCC with its orbit partition, and only pages in the same orbit share a key.

Suppose that a user wants to navigate our website and has obtained the hash of one specific page. The browser retrieves the index and SCC key from the DHT, then retrieves the canonical form of the quotient, from which the quotient digraph can be reconstructed. The index points to a specific node of the quotient, whose contents the browser displays. If the user clicks a link within the same SCC, the browser follows the corresponding edge of the quotient. Since the quotient preserves ordered unfoldings (Proposition 5.3), the user sees exactly the sequence of pages they would see on the original site. For links that leave the SCC, the embedded hash is a page key, and the browser repeats the process from the beginning. As with any Merkle structure, editing a page changes the keys of that page's SCC and of all pages from which it can be reached, and of no others.

8 Related Work

Merkle hashing of cyclic definitions. The core idea of the digraph Merkle hashing algorithm was independently discovered by the developers of the Unison programming language [2], who hash the abstract syntax trees of mutually recursive definitions. Each function and datatype is a node in a digraph, with its AST as the node label, and each cycle of mutually recursive definitions is hashed as a unit. To our knowledge their scheme does not treat non-trivial automorphism orbits inside a cycle, which arise, for example, from two structurally identical mutually recursive definitions that differ only in name. Such orbits appear to be rare in practice, but a complete treatment requires either the orbit index of Algorithm 3.2 or the local variant of Section 6, in which non-trivial orbits never arise (Lemma 6.1). Hash-consing of cyclic structures and unique representations of regular trees [21], as well as hashing of terms modulo α-equivalence [22], address closely related problems in programming language implementation.

Color refinement and Weisfeiler–Leman. The Weisfeiler–Lehman graph hash [35] iteratively hashes and aggregates the neighborhoods of each node of an undirected graph. Isomorphic graphs are guaranteed to receive equal hashes, but the converse fails: 1-dimensional Weisfeiler–Leman refinement [38] cannot distinguish, for example, two disjoint triangles from a 6-cycle, or any two regular graphs of the same order and degree. In the kernel literature the number of iterations h is a parameter that controls the depth of the subtree features. When refinement is used to test or hash equivalence, as here, no parameter is needed, because the partition stabilizes within $n-1$ rounds (Lemma 4.4). Efficient implementations of color refinement run in $O((n+m)\log n)$ time [11, 8]. The algorithm of Section 4 is a directed, canonical variant of the Weisfeiler–Lehman hash. The equivalence it computes is the view equivalence of anonymous networks [39] and the fibration equivalence of Boldi and Vigna [10], and for undirected graphs it corresponds to isomorphism of universal covers [27].

Bisimulation and automata. Bisimilarity is computed by relational coarsest partition algorithms [16, 30], and state equivalence of deterministic automata by Hopcroft-style minimization [14, 18, 37]. Sections 4–6 use these as the first stage of canonical hashing.

Canonization. Canonical labeling tools [23, 24, 15, 12] and Babai's quasipolynomial-time isomorphism test and canonization [6, 7] underlie Section 3.

Arshad et al. In a 2014 paper, Arshad et al. [5] presented an algorithm for hashing directed graphs given additional inputs: a fixed source node and a depth-first search traversal tree. The traversal tree classifies the edges as tree, back, forward, and cross edges depending on search order, and the hash uses the traversal numbers to order the nodes. The authors note that isomorphic graphs may hash to different values because of this dependence on the source node and DFS tree. Their approach can be used to hash nodes with ordered outgoing edges and can be viewed as an alternative to Algorithm 5.1.

In a separate 2018 paper, Arshad et al. [4] presented a graph hashing algorithm in which the hash of a node combines, by XOR, the hashes of the labels of all of that node's descendants; the graph hash is then computed by hashing each node together with an accumulated graph hash. As described, this approach has two weaknesses. First, XOR aggregation is not a multiset hash: an even number of identically labeled descendants cancels out, so a node with two identically labeled descendants contributes as if it had none. Second, the aggregation ignores the edge structure among the descendants. For example, in the directed k-cycle and in the complete digraph on k vertices, with all labels equal, every node has the same $k-1$ identically labeled descendants, so the algorithm outputs equal hashes for these non-isomorphic graphs.

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

Table 9.1: Validation against brute-force equivalence checks: number of node pairs (or SCCs) compared, with no disagreements in any row.

Scheme (theorem)	Pairs checked
Global graph hash (Thm. 3.1)	1,000 graphs
Global node hash (Thm. 3.2)	12,947
Local hash (Thm. 4.11)	12,947
Bisimulation hash (Thm. 4.14)	12,947
Ordered node hash (Thm. 5.2)	12,720
Ordered local hash (Prop. 5.3)	12,720
Ordered SCC orbits (Thm. 5.4)	2,487 SCCs
Merkle hash, global variant (Thm. 6.4)	26,794
Local component hash (Thm. 6.2)	78,594
Merkle hash, local variant (Thm. 6.4, Prop. 6.5)	78,594

9 Experimental Results We implemented all algorithms of this paper in Python 3.11 as an exact reference implementation, using SHA-256 over a length-prefixed injective encoding. Global canonization uses a simple exhaustive individualization-refinement search, adequate for small graphs. Running times were measured on a single core of an Intel Xeon processor at 2.1 GHz. The implementation is intended to validate the theorems, not to compete with optimized tools.

Validation. We generated 1000 pairs of random labeled digraphs with 1–6 nodes each (edge probability 0.15, 0.3 or 0.5, self-loops allowed, one or two labels). In half of the pairs the second graph is a random relabeling of the first. For every pair of nodes we compared hash equality with an independent brute-force decision of the corresponding equivalence:

- isomorphism and rooted isomorphism, by enumerating all bijections;
- unfolding isomorphism, by comparing canonical forms of truncated unfoldings of depth $n_1 + n_2 + 1$;
- bisimilarity, by a greatest-fixpoint computation;
- rooted ordered isomorphism, by simultaneous traversal.

For $\approx_{\mathrm{glob}}$ we used a direct implementation of Definition 6.3. For the local variant we ran a second batch of 3000 unordered and 3000 ordered pairs (1–6 nodes, edge probability 0.15, 0.3 or 0.45, a random relabeling in 30% of the pairs, and, in ordered graphs, occasional repeated links to the same target). There we compared the local component hash with unfolding isomorphism on every pair of SCCs, and the local Merkle hash with isomorphism of SCC-annotated unfoldings (truncated at depth $2(n_1 + n_2) + 2$); we also checked the implications of Proposition 6.6 and its separating examples. We also tested the ordered orbit partition of Theorem 5.4 against brute-force search on every SCC. Table 9.1 summarizes the results. No disagreement was found. The same harness confirms the counterexamples of Examples 4.13 and 5.1 for the naive variants, and the orbit structure claimed in Figure 2.1.

Scaling. Table 9.2 reports median running times over three random ordered digraphs per size, with two labels and $2n$ randomly drawn edges (duplicates discarded; average out-degree just under 2). These graphs have a giant strongly connected component containing 60–65% of the nodes. Global stabilization was detected after 5–6 refinement rounds at all sizes. For all schemes except the local Merkle variant, the time to hash all nodes grows roughly by a factor of four each time n doubles, i.e. quadratically. For $\mathcal{H}_L$, $\mathcal{H}_B$ and $\mathcal{H}_O$ this matches the output-sensitive term $\sum_x |R(x)|$ discussed in Section 4: most nodes reach the giant component. The cubic worst case is not attained because few rounds are needed. The global Merkle variant $\mathcal{H}_M^{\mathrm{glob}}$ is cheaper than hashing every node with $\mathcal{H}_O$, because the depth-first searches are confined to each SCC, but it is still quadratic in the size of the giant component.

The local Merkle variant $\mathcal{H}_M^{\mathrm{loc}}$ is one to two orders of magnitude faster, and its running time roughly doubles when n doubles. Each component quotient is serialized once, and only a few refinement rounds are needed. On

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

Table 9.2: Median time in seconds to hash all nodes of a random ordered digraph with n nodes and $2n$ edges (Python prototype). The last two columns use the local Merkle variant, treating the graph as ordered and as unordered, respectively.

n	largest SCC	$\mathcal{H}_L$	$\mathcal{H}_B$	$\mathcal{H}_O$	ordered local	$\mathcal{H}_M^{\text{glob}}$ (ord.)	$\mathcal{H}_M^{\text{loc}}$ (ord.)	$\mathcal{H}_M^{\text{loc}}$ (unord.)
250	151	0.27	0.23	0.10	0.10	0.07	0.009	0.010
500	314	0.92	0.97	0.39	0.40	0.26	0.020	0.021
1000	646	3.84	3.44	1.83	1.63	1.20	0.042	0.046
2000	1272	17.80	14.69	6.62	5.89	4.43	0.084	0.091

the same kind of graphs with $n = 64{,}000$ nodes (largest SCC about 40,500 nodes) it takes about 5.2 s (ordered) and 5.4 s (unordered).

Granularity. To illustrate how the equivalences compare, we hashed 400 random unlabeled digraphs with 12 nodes and 18 randomly drawn edges (duplicates discarded) and counted the distinct hash values per graph. On average the graphs have 12 nodes and the following numbers of classes:

- 11.82 orbits (global node hash);
- 9.81 $\approx_{\text{glob}}$ classes (global Merkle hash);
- 9.66 $\approx_{\text{loc}}$ classes (local Merkle hash);
- 9.13 unfolding classes (local hash);
- 7.76 bisimulation classes.

This is consistent with the implications rooted isomorphism $\Rightarrow \approx_{\text{glob}} \Rightarrow \approx_{\text{loc}} \Rightarrow$ unfolding isomorphism $\Rightarrow$ bisimilarity (Proposition 6.6 and Section 4). The gaps show that the choice of equivalence matters in practice even on small random graphs.

10 Conclusion We have given hashing schemes for node-labeled digraphs whose hash equality corresponds exactly, up to collisions of the underlying string hash, to isomorphism, rooted isomorphism, unfolding isomorphism, bisimilarity, and two Merkle-style equivalences, together with fast variants for graphs with ordered outgoing edges. The local Merkle variant combines the incrementality of Merkle hashing with polynomial running time, and it is near-linear in our experiments. Natural directions for future work include an $O((n+m)\log n)$ implementation of the local stage via partition refinement, output-sensitive or incremental computation of local hashes, polynomial canonization of general ordered digraphs, and evaluation on real data such as computer program call graphs and web crawls.

Acknowledgements Early versions of this paper were posted on arXiv in 2020, and some preliminary results were presented at the 51st Southeastern International Conference on Combinatorics, Graph Theory, and Computing (SEICCGTC). These results were not published in the proceedings and were not peer reviewed. If you are looking for an earlier version of the paper, see the arXiv page at arXiv:2002.06653.

Per SIAM's policy on AI disclosure, we disclose that we utilized Claude Opus 5.5 to transform proof sketches to full proofs in Section 4 and Section 6. Opus 5.5 was utilized to simplify a human written proof to the one seen in Theorem 3.2. Opus 5.5 was used to tidy up other sections of the paper and as simulated peer review. The core algorithms presented and the code implementations have been under development by the author of this paper since 2015, and this paper is a product of many years of human effort.

References

[1] *IPFS is the distributed web*, https://ipfs.io/. Accessed: 2019-12-09.

[2] *Unison programming language*, https://www.unisonweb.org/. Accessed: 2019-12-09.

[3] A. V. Aho, J. E. Hopcroft, and J. D. Ullman, *The Design and Analysis of Computer Algorithms*, Addison-Wesley, 1974.

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

[4] M. U. Arshad, A. Kundu, E. Bertino, A. Ghafoor, and C. Kundu, *Efficient and scalable integrity verification of data and query results for graph databases*, IEEE Trans. Knowl. Data Eng., 30 (2018), pp. 866–879.

[5] M. U. Arshad, A. Kundu, E. Bertino, K. Madhavan, and A. Ghafoor, *Security of graph data: hashing schemes and definitions*, in Proceedings of the 4th ACM conference on Data and application security and privacy, CODASPY '14, New York, NY, USA, Mar. 2014, Association for Computing Machinery, pp. 223–234.

[6] L. Babai, *Graph isomorphism in quasipolynomial time [extended abstract]*, in Proceedings of the Forty-Eighth Annual ACM Symposium on Theory of Computing, STOC '16, New York, NY, USA, 2016, Association for Computing Machinery, p. 684–697, https://doi.org/10.1145/2897518.2897542, https://doi.org/10.1145/2897518.2897542.

[7] L. Babai, *Canonical form for graphs in quasipolynomial time: preliminary report*, in Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing, STOC 2019, New York, NY, USA, 2019, Association for Computing Machinery, p. 1237–1246, https://doi.org/10.1145/3313276.3316356, https://doi.org/10.1145/3313276.3316356.

[8] C. Berkholz, P. Bonsma, and M. Grohe, *Tight lower and upper bounds for the complexity of canonical colour refinement*, Theory of Computing Systems, 60 (2017), pp. 581–614.

[9] N. Bloyet, P.-F. Marteau, and E. Frénod, *Scott: A method for representing graphs as rooted trees for graph canonization*, in Complex Networks and Their Applications VIII, Studies in computational intelligence, Springer International Publishing, Cham, 2020, pp. 578–590.

[10] P. Boldi and S. Vigna, *Fibrations of graphs*, Discrete Math., 243 (2002), p. 21–66, https://doi.org/10.1016/S0012-365X(00)00455-6, https://doi.org/10.1016/S0012-365X(00)00455-6.

[11] A. Cardon and M. Crochemore, *Partitioning a graph in $O(|A| \log_2 |V|)$*, Theoretical Computer Science, 19 (1982), pp. 85–98.

[12] P. T. Darga, M. H. Liffiton, K. A. Sakallah, and I. L. Markov, *Exploiting structure in symmetry detection for CNF*, in Proceedings of the 41st annual Design Automation Conference, ACM, 2004, pp. 530–534.

[13] M. Hennessy and R. Milner, *Algebraic laws for nondeterminism and concurrency*, J. ACM, 32 (1985), p. 137–161, https://doi.org/10.1145/2455.2460, https://doi.org/10.1145/2455.2460.

[14] J. Hopcroft, *An n log n algorithm for minimizing states in a finite automaton*, in Theory of Machines and Computations, Z. Kohavi and A. Paz, eds., Academic Press, 1971, pp. 189–196, https://doi.org/10.1016/B978-0-12-417750-5.50022-1, https://www.sciencedirect.com/science/article/pii/B9780124177505500221.

[15] T. Junttila and P. Kaski, *Engineering an efficient canonical labeling tool for large and sparse graphs*, in 2007 Proceedings of the Ninth Workshop on Algorithm Engineering and Experiments (ALENEX), SIAM, 2007, pp. 135–149.

[16] P. C. Kanellakis and S. A. Smolka, *Ccs expressions, finite state processes, and three problems of equivalence*, (1983), p. 228–240, https://doi.org/10.1145/800221.806724, https://doi.org/10.1145/800221.806724.

[17] J.-Y. Kao, N. Rampersad, and J. Shallit, *On nfas where all states are final, initial, or both*, Theor. Comput. Sci., 410 (2009), p. 5010–5021, https://doi.org/10.1016/j.tcs.2009.07.049, https://doi.org/10.1016/j.tcs.2009.07.049.

[18] T. Knuutila, *Re-describing an algorithm by hopcroft*, Theor. Comput. Sci., 250 (2001), pp. 333–363.

[19] D. Kőnig, *Theorie der endlichen und unendlichen Graphen*, Akademische Verlagsgesellschaft, Leipzig, 1936.

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

[20] J. Loeliger and M. McCullough, *Version Control with Git: Powerful Tools and Techniques for Collaborative Software Development*, "O'Reilly Media, Inc.", Aug. 2012.

[21] L. Mauborgne, *An incremental unique representation for regular trees*, Nordic J. of Computing, 7 (2000), p. 290–311.

[22] K. Maziarz, T. Ellis, A. Lawrence, A. Fitzgibbon, and S. Peyton Jones, *Hashing modulo alpha-equivalence*, in Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation, PLDI 2021, New York, NY, USA, 2021, Association for Computing Machinery, p. 960–973, https://doi.org/10.1145/3453483.3454088, https://doi.org/10.1145/3453483.3454088.

[23] B. D. McKay, *Practical graph isomorphism*, Department of Computer Science, Vanderbilt University Tennessee, USA, 1981.

[24] B. D. Mckay and A. Piperno, *Practical graph isomorphism, ii*, J. Symb. Comput., 60 (2014), p. 94–112, https://doi.org/10.1016/j.jsc.2013.09.003, https://doi.org/10.1016/j.jsc.2013.09.003.

[25] R. C. Merkle, *A digital signature based on a conventional encryption function*, in Conference on the theory and application of cryptographic techniques, Springer, 1987, pp. 369–378.

[26] S. Nakamoto, *Bitcoin: A peer-to-peer electronic cash system*, (2008).

[27] N. Norris, *Universal covers of graphs: Isomorphism to depth $n-1$ implies isomorphism to all depths*, Discrete Applied Mathematics, 56 (1995), pp. 61–74.

[28] N. I. of Standards and Technology, *Secure hash standard*, Tech. Report Publication 180-2, 100 Bureau Drive, Stop 8900, Gaithersburg, MD 20899-8900, August 2002, https://csrc.nist.gov/csrc/media/publications/fips/180/2/archive/2002-08-01/documents/fips180-2.pdf.

[29] N. I. of Standards and Technology, *SHA-3 standard: Permutation-based hash and extendable-output functions*, Tech. Report Publication 202, 100 Bureau Drive, Stop 8900, Gaithersburg, MD 20899-8900, August 2015, https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.202.pdf.

[30] R. Paige and R. E. Tarjan, *Three partition refinement algorithms*, SIAM Journal on Computing, 16 (1987), pp. 973–989, https://doi.org/10.1137/0216062, https://doi.org/10.1137/0216062, https://arxiv.org/abs/https://doi.org/10.1137/0216062.

[31] C. Picard and R. Matthes, *Coinductive graph representation: the problem of embedded lists*, Electronic Communications of the EASST, 39 (2011), https://doi.org/10.14279/tuj.eceasst.39.649, https://eceasst.org/index.php/eceasst/article/view/2664.

[32] R. Rivest, *The MD5 message-digest algorithm*, Tech. Report RFC 1321, MIT Laboratory for Computer Science and RSA Data Security, Inc., 545 Technology Square, Cambridge, MA 02139-1986, April 1992, https://tools.ietf.org/html/rfc1321.

[33] D. Sangiorgi, *Introduction to Bisimulation and Coinduction*, Cambridge University Press, 2011.

[34] N. Shervashidze and K. Borgwardt, *Fast subtree kernels on graphs*, in Advances in Neural Information Processing Systems, Y. Bengio, D. Schuurmans, J. Lafferty, C. Williams, and A. Culotta, eds., vol. 22, Curran Associates, Inc., 2009, https://proceedings.neurips.cc/paper_files/paper/2009/file/0a49e3c3a03ebde64f85c0bacd8a08e2-Paper.pdf.

[35] N. Shervashidze, P. Schweitzer, E. J. Van Leeuwen, K. Mehlhorn, and K. M. Borgwardt, *Weisfeiler-Lehman graph kernels*, J. Mach. Learn. Res., 12 (2011).

[36] R. Tarjan, *Depth-first search and linear graph algorithms*, SIAM Journal on Computing, 1 (1972), pp. 146–160, https://doi.org/10.1137/0201010, https://doi.org/10.1137/0201010, https://arxiv.org/abs/https://doi.org/10.1137/0201010.

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

[37] A. Valmari and P. Lehtinen, *Efficient minimization of DFAs with partial transition functions*, in Proceedings of the 25th International Symposium on Theoretical Aspects of Computer Science (STACS), vol. 1 of LIPIcs, 2008, pp. 645–656.

[38] B. Weisfeiler and A. Leman, *The reduction of a graph to canonical form and the algebra which appears therein*, Nauchno-Technicheskaya Informatsia, Series 2, 9 (1968), pp. 12–16. English translation available.

[39] M. Yamashita and T. Kameda, *Computing on anonymous networks. i. characterizing the solvable cases*, IEEE Transactions on Parallel and Distributed Systems, 7 (1996), pp. 69–89, https://doi.org/10.1109/71.481599.

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited